

QUARKUS BUILD DAY

Beginner Learning Pack

Understand the framework. Run the service. Make your first change.

Lloyds Banking Group × DevOps Society

BEGINNER FRIENDLY

What you will learn

A practical introduction with a clear route from concepts to hands-on exploration.

01

Describe Quarkus

Explain what the framework is and where it fits.

02

Connect the concepts

Relate microservices, APIs, containers and cloud-native delivery.

03

Use Dev Mode

Run the service and see live coding in action.

04

Build confidence

Explore, change and extend a real application safely.

No prior Quarkus experience is required.

What is Quarkus?

A Java framework designed for fast, efficient, cloud-native applications.

Fast developer feedback

Dev Mode supports live coding, so changes appear without repeatedly restarting the application.

Cloud-native by design

Quarkus is suited to REST services, containers and platforms such as Kubernetes.

Production-aware

Health, metrics, configuration, logging and testing form part of the wider engineering story.



From monolith to microservices

A microservice focuses on one capability and communicates with other services through APIs.



Smaller scope

Each service owns a focused responsibility.

Independent change

Teams can evolve a service without rebuilding one huge application.

Clear interfaces

REST APIs provide a predictable way for systems to communicate.

DISCUSS

Where have you seen small services or APIs used before?

Why Quarkus for modern Java?

The value is not just speed. It is a shorter, more observable path from idea to running service.



Fast startup

Useful when services need to start or scale quickly.



Low memory

Efficient applications can make better use of platform resources.



Live coding

Tight feedback loops make experimentation easier.



Cloud-native

Designed to work well with containers and Kubernetes.



Engineering ready

Supports health, testing, configuration and observability patterns.



Delivery focused

Fits build, image and deployment workflows.

Prerequisites and dependencies

Use the project's wrapper and supplied repository where possible to reduce setup friction.

ESSENTIAL

Java

JDK 17 or newer

IDE

IntelliJ IDEA or VS Code

Git

Clone and explore the repository

Build tool

Maven 3.9+ or the supplied Maven/Gradle wrapper

ACTIVITY DEPENDENCIES

Docker

Needed for the container and supporting-service activities

Browser / API client

Browser, Bruno or Postman for calling endpoints

Basic knowledge

Java classes and methods; REST awareness helps

Workshop project

Pre-cloned if corporate network access may be restricted

Quick check: `java --version` • `mvn --version` • `docker --version` • `git --version`

Anatomy of a Quarkus project

You do not need to memorise every file. Start by learning where the main responsibilities live.

REST resource



Receives HTTP requests and returns responses.

Service / bean



Holds reusable logic and supports dependency injection.

Configuration



application.properties stores environment-aware settings.

Tests



Validate endpoint behaviour and application logic.

Build file



pom.xml or build.gradle manages extensions and build tasks.

Dev UI

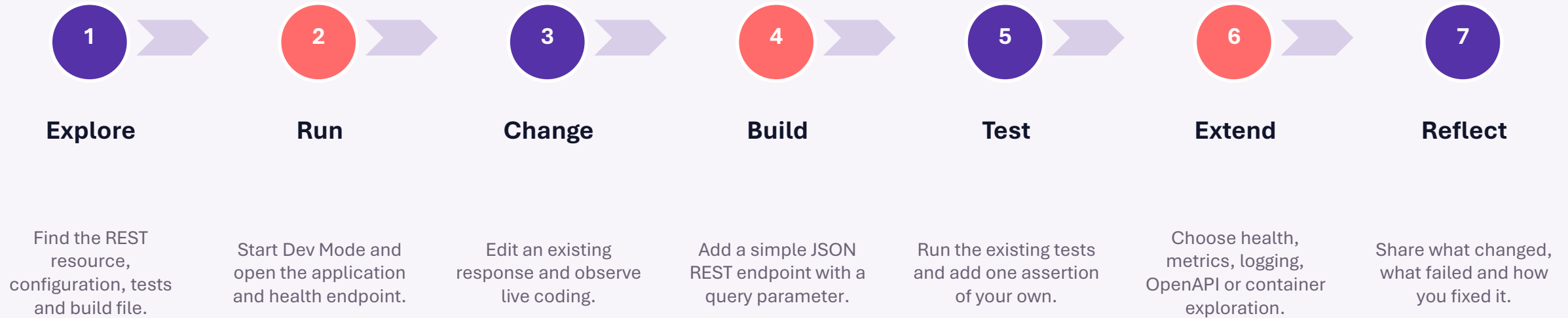


A developer-focused view of installed features and tooling.

Beginner habit: follow one request from endpoint → logic → response → test.

Activity pathway

Work in pairs. Complete the core activities first, then select side quests if time allows.



CORE OUTCOME A running Quarkus service, one code change, one new endpoint and one passing test.

Your first change in Dev Mode

The aim is to experience the feedback loop, not to write lots of code.

1 Start the app

```
./mvnw quarkus:dev
```

```
# or, for a Gradle project
```

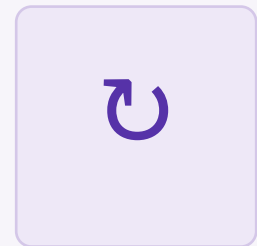
```
./gradlew --console=plain quarkusDev
```

2 Open the endpoint

```
http://localhost:8080/hello
```

3 Change the greeting and refresh

- Find GreetingResource
- Edit the returned text
- Save the file
- Refresh the browser
- Describe what happened



Why is this useful to a developer?

Side quests

Pick one based on confidence and available time. There is no need to complete everything.

BRONZE

Explore

Open Dev UI, health and API documentation.



SILVER

Observe

Add a log entry; inspect metrics and configuration.



GOLD

Extend

Add validation, a new endpoint or persistence.



PLATINUM

Package

Build a container image and explain the deployment path.



Facilitator tip: match the challenge to the learner, not the learner to the challenge.

Reflection and discussion

Use the final ten minutes to connect the activity to production engineering.

Reliability

How would we know the service is healthy?

Observability

Which logs and metrics would help us diagnose a problem?

Security

What needs protecting before an API is exposed?

Scalability

What changes when usage grows?

Delivery

How does code become a tested, deployable image?

Learning

What was unfamiliar, and what now makes sense?

Share one insight • one question • one next step

You are ready to build.

Explore → Run → Change → Build → Test → Extend → Reflect

**Beginner success is not completing every task.
It is understanding one service well enough to change it with
confidence.**

Questions? Pair up, experiment, and ask for help early.